

1강 - 수업

인터넷

장치(devices) : 수억개가 연결되어 있음

hosts(호스트) : **end systems**

edge에서는 네트워크 앱(크롬 등..)들이 동작중임

Packet switches : 패킷을 전달하는 것

Packets: chunks of data 데이터 뭉텅이

routers, switches

Communication Link

: 섬유, 구리, radio, 위성

bandwidth : 전송 속도(transmission rate)

Networks

devices, routers, links들의 집합체. 각각의 조직에 의해서 관리됨. ex) 대학 넷, 지역 넷, 콘텐츠 제공자 넷...

Internet

네트워크의 네트워크 - 서로연결된 ISPs(인터넷 서비스 제공자; Internet Service Provider)

Protocols

규약, 약속 -> 어떤 메시지를 보내야하고 받아야할지에 대한 규칙

e.g. HTTP, 동영상 스트리밍, Skype, TCP, IP, Wifi, 4G, 5G, 이더넷...

Internet Standards

RFC(Request for Comments) 문서에 규정되어 있음.

by IETF(Internet Engineering Task Force)에 의해서 규정되고 있음

Infrastructure

: 어플리케이션 서비스를 제공하는 인프라

웹, 동영상 스트리밍, 화상회의, 이메일, 게임....

Programming interface

: 프로그래밍 인터페이스를 이용하면 개발을 할 수 있다.

Protocol 프로토콜이 무엇인가?

인간 프로토콜 : "몇시야?" "질문이 있어"...

네트워크 프로토콜 : 프로토콜에 의해 지배되는 네트워크에서의 모든 커뮤니케이션

프로토콜 : 네트워크 개체 간에 송수신되는 메시지의 형식, 순서, 메시지 전송, 수신 시 수행되는 작업의 정의.

특정 보내진 메시지

특정 행동 when 메시지 받거나 이벤트가 있을 때

1강 - 실습

컴퓨터 네트워크란?

: 상호 연결된 장치 그룹, 리소스, 정보, 데이터를 공유하기 위해 서로 통신하는 것.
호스트 + 커뮤니케이션 링크 + 메시지

Host(End system) : 네트워크 안에 있는 하드웨어 components. Computer, 스마트폰, IoT...
Communication link : Wired(유선), Wireless(Wifi, 4G/5G, etc..)
Message(Packet) : 통신하는 Data

인터넷이란?

컴퓨터 네트워크의 특정 부분
애플리케이션의 서비스를 제공하는 infrastructure

Protocol 프로토콜이란?

- * 네트워크를 통해 데이터가 전송되는 방식을 정의하는 규칙(rules)
- * 두 개 이상의 통신 개체 간에 교환되는 메시지의 형식 및 순서
- * 메시지 또는 기타 이벤트의 전송 및 수신에 대한 액션
- * 우리가 보내는 특정 메시지와 수신된 답변에 대한 응답으로 취하는 특정 조치

2강 수업 - 네트워크 엣지와 코어

Network Edges

Hosts : Clients와 Servers
Server는 주로 데이터 센터에 있음

Access networks, Physical media

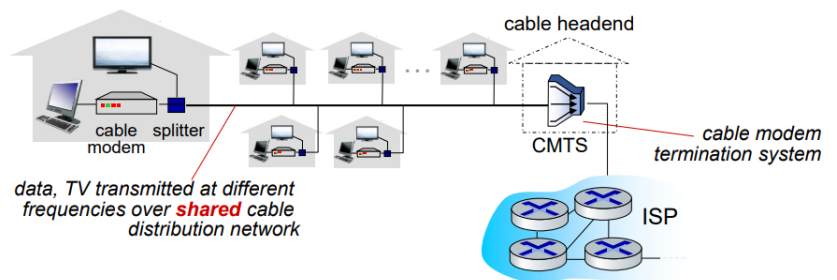
유선 또는 무선의 communications link를 가지는 네트워크.
근거리에서 서로 연결된 디바이스들의 네트워크 ex) 홈 네트워크

Network Core

서로연결된 routers
네트워크의 네트워크

end systems를 엣지 라우터에 어떻게 연결할까?

지역 access network
단체(학교, 회사) access network
모바일 access network : WiFi, 4G, 5G



Cable-based access 케이블 기반 액세스

FDM(Frequency Division Multiplexing) : 각각 다른 주파수 대역(밴드)을 통해서 다른 채널을 전송한다.
ex) 1~5번 채널에는 비디오를 보내고, 7~8번 채널에는 Data(인터넷)를 보냄

Cable headend(헤드엔드) : 분배장치, 지역 사무소, 주 조종장치, 마스터 기기

CMTS : Cable Modem Termination System. 케이블로 공유된 라인 사용. 자신에게 맞는 것만 받아서 사용.
Access Network를 공유한다!

HFC(Hybrid Fiber Coax) : 동+광케이블 하이브리드.

Assymetric(비대칭적)임. downstream transmission rate 40Mbps- 1.2Gbps

 upstream transmission rate 30-100Mbps

=> 사용자가 다운로드 하는게 훨씬 많기 때문

Cabel, Fiber는 가정을 ISP router에 연결시켜 줌

가정은 **access network**를 cable headend에 공유함.

Metric Prefix 접두어 정리

10 ^{^3}	Kilo	k	10 ^{^-2}	centi	c
10 ^{^6}	Mega	M	10 ^{^-3}	milli	m
10 ^{^9}	Giga	G	10 ^{^-6}	micro	μ
10 ^{^12}	Tera	T	10 ^{^-9}	nano	n

1Gbps = 1000Mbps

bps : bit per second 초당 비트수

0.3Mbps = 300kbps

Digital Subscriber Line(DSL)

central office로부터 단독(dedicated) line을 가지고 있다. 전화, 데이터 다른 주파수를 통해 전달.

이미 존재하는 전화선을 Central office DSLAM에 연결

data는 DSL전화선을 통해 인터넷으로

voice는 DSL 전화선을 통해 전화망으로

24-52Mbps 지정된 upstream transmission rate

<- Cable based보다 더 느림!

3.5-16Mbps 지정된 downstream transmission rate

Home networks

유선 이더넷 - 1Gbps

와이파이 무선 - 54, 450Mbps

Wireless Access Networks 무선 액세스 네트워크

AP(Access Point) : 무선 공유기

WLAN(Wireless Local Area Networks) :

특정 지역이나 건물 주변 (~100 ft)

802.11b/g/n (WiFi): **11, 54, 450Mbps** transmission rate

Wide-area Cellular Access Networks :

모바일, 셀룰러 네트워크 제공자 (10s KM)

10s Mbps

4G/5G 셀룰러 네트워크

Enterprise networks (기업망)

기업, 대학에서 사용

wired, wireless 혼용. 스위치, 라우터 혼용

Ethernet : wired access at 100Mbps, 1Gbps, 10Gbps

Wifi : WiFi AP at 11, 54, 450Mbps

Data Center Networks

데이터센터같은 곳

고대역폭 링크 High-bandwidth links **10~100s Gbps**)

수백, 수천개의 서버와 인터넷을 연결

Links : 물리적 media(매체)

bit : 전송자-수신자 사이에서 전파되는 것. 0과 1 8bit=1byte

physical link : 물리적인 연결선

guided media(유도 매체) : 고체 media를 통해 전파 - 구리, 섬유, 동

unguided media(비유도 매체) : 자유롭게 전파되는 신호 - 라디오

TP(Twisted Pair) :

두 개의 절연된(insulated) 구리선으로 이루어진 줄

Cat. 5 : 100Mbps, 1Gbps 이더넷

Cat. 6 : 10Gbps 이더넷

Coaxial Cable 동축 케이블:

두 동심원(concentric) 구리 도체

bidirectional 양방향

broadband(광대역) : 케이블 내에 다양한 주파수 채널이 있다. 각 채널마다 100Mbps.

Fiber Optic cable 광케이블:

유리섬유가 빛 pulse를 보냄. 각각의 pulse가 비트임.

고속 전송 가능 : 10s~100sGbps

낮은 에러율 : 리피터를 줄일 수 있음. 전자기적 electromagnetic 노이즈에 면역임.

Wireless Radio 무선 라디오:

다양한 대역에서 전자기파 스펙트럼을 통해 전송되는 신호

물리적인 줄 없음

broadcast(일대다 전송), half-duplex(한 번에 한 방향만 전송 가능)

환경이 영향을 끼친다. 반사, 물체에 의한 방해, 간섭/노이즈 - 낮은 주파수는 회절 가능, 높은 주파수는 직진

Radio Link의 종류

Wireless LAN(WiFi) : 10~100Mbps; 10s 미터 범위

Wide-Area(4G/5G) : 10s Mbps; 10km까지 범위

Bluetooth : 줄 대체. 단거리, 제한된 속도

Terrestrial microwave(지상파) : point-to-point 45Mbps

Satellite 위성 : 최대 100Mbps(Starlink) 다운링크. 270msec의 딜레이. (geostationary; 정지궤도)

2강 - 실습 HTML

HTML : Hyper Text Markup Language.

웹 콘텐츠의 표준

<tags>를 이용해 웹 페이지의 구조를 저으입니다.

<tags>들이 화면에 뭘 어떻게 띄울지를 구성한다. including 텍스트, 이미지, 링크, 미디어....

Elements

- **Building blocks** of an HTML document, typically defined using tags like <h1>, <p>, and <a>

Tags

angle brackets <>에 둘러싸여 있음. <html> s<body> <div> <h1>
<> - 열기 </> - 닫기

Attributes

element에 대한 추가 정보. 동작이나 모습 customise하기 위해서 사용

Semantic Tags

태그의 이름만으로 그 안에 무슨 내용이 포함되어 있는지 의미를 명확히 알 수 있음

Readability(가독성) 좋게 함.

SEO(Search Engine Optimization) : 엔진이 더 쉽게 해석하고 랭크할 수 있음.

Accessibility(접근성) : 스크린 리더 같은 보조 페이지 장애가 있는 사용자들도 구조를 이해할 수 있게 함.

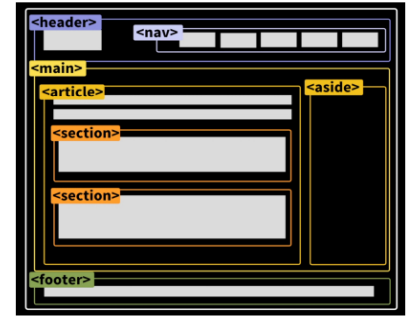
<header> : header 섹션을 정의. logos, navigation links, titles 포함.

<nav> : navigation links 페이지를 표현한다.

<main> : 페이지의 메인 콘텐츠를 담음.

<article>, <section>, <footer>, <aside> etc...

- Non-Semantic Tags 의미없는 태그들(for comparison): <div>, , etc...



3강 수업 - 네트워크 코어

Host : Sends packets of data 패킷단위로 데이터 전송

Application message를 보낸다!

Packet이라는 작은 chunk로 나누어 보낸다. L bits 길이.

Transmission rate(aka link capacity, link bandwidth)가 R 인 Access Network를 통해서 보낸다.

$$R = \frac{L}{T} \begin{matrix} \text{Transfer bit} \\ \text{Transfer time} \end{matrix}$$

예: 2Mbit data, 10s
 $\rightarrow R = \frac{2000\text{kbit}}{10\text{s}} = 200\text{kbps}$

Packet transmission delay(패킷전송시간) = 링크를 통해 L 비트 패킷을 보내는데 걸리는 시간
 $= L(\text{bits; 양}) / R(\text{bps; 시간})$

예: 20kbit packet, 1Mbps Ethernet line,
 $\rightarrow \text{delay} = \frac{20\text{kbit}}{1000\text{kbit/s}} = 0.02\text{s} = 20\text{ms}$

다양한 Wireless Radio

Radio wave

Low Frequency (3KHz to 100GHz)

회절(diffraction)이 잘 일어남

FM라디오 87.5~108.0MHz

이동통신 주파수 : 1.8, 2.1, 3.5GHz

WiFi 주파수 : 2.4, 5GHz

블루투스 주파수 : 2.4GHz

Microwave

High Frequency (1GHz to 1000GHz)

직진성 Straight

Terrestrial microwave(지상파)

Satellite communication(위성통신)

Network Core 네트워크 코어

Packet-Switching :

hosts가 application-layer의 메시지를 packets으로 쪼개고 네트워크가 다음 router로 보내는 것.

Source에서 Destination으로

가변길이의 메시지를 고정길이의 packet으로 쪼갬

두 가지 네트워크 코어의 기능

1. Forwarding (aka Switching)

Local action : 라우터의 input link로 들어오는 패킷을 output link로 적절하게 옮기는 것
패킷 헤더(메타데이터)의 목적지 주소를 보고 가는 길을 결정함.

2. Routing 라우팅

Global action : Source-Destination 경로를 결정하는 것

Packet Switching의 두 가지 방식

1. Store-and-forward

: 다음 링크로 보내지기 전에 전체 패킷이 반드시 라우터에 도착해야 한다.

Packet transmission delay : L/R 초가 걸린다. L 비트의 패킷 R bps의 속도

ex) $L=10\text{Mbits}$, $R=100\text{Mbps}$

one-hop(하나 움직이는데) trans delay = 0.1sec

2. Queueing

: 작업이 서비스되는 것보다 작업이 더 빨리 도착하면 일어난다.

Packet queuing and loss : arrival rate(도착률)가 transmission rate(전송률)을 넘어서면

* 패킷이 대기열에 들어가 출력될 때까지 기다린다

* 라우터의 메모리(buffer)가 가득 차면 drop(lost)된다.

>>중간에 lost되면 다시 보내달라고 하면 됨.

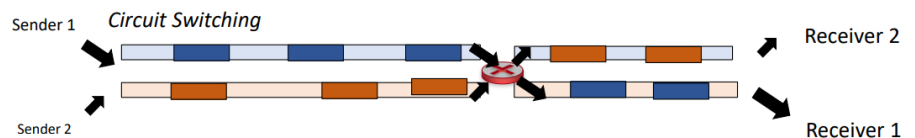
Circuit Switching : Packet switching 이전

source와 destination을 연결하기 위해 "call"을 해서 예약하고 end-to-end로 할당한다.

지정된 리소스. 독점(dedicated resource) : 자원을 공유할 수 없음. 회로같은(보장된) 성능을 보여줌

전화를 사용하지 않으면 다 idle 상태가 됨. 비효율적

전통적인 전화 네트워크에서 사용됨.



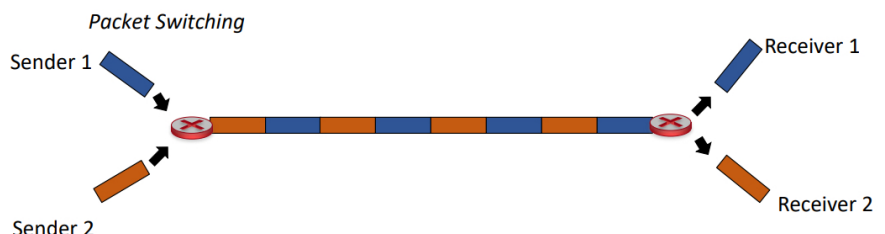
Circuit Switching vs Packet Switching

Q)서킷과 패킷에서 얼마나 많은
사용자가 이용할 수 있을까?

1Gb/s link의 회선

각각의 user는 active일 때 100Mbps

전체 시간의 10%만 활동



Circuit Switching : 10명

Packet Switching : 35명이 같이 쓸 때, 10명이상 active 될 확률은 0.004% >> 암튼 Packet이 훨씬 효율적

Q) 그렇다면 항상 패킷 스위칭이 좋은가?

간헐적인(bursty) 데이터에는 좋다. 자원 공유. call setup을 할 필요 없음.

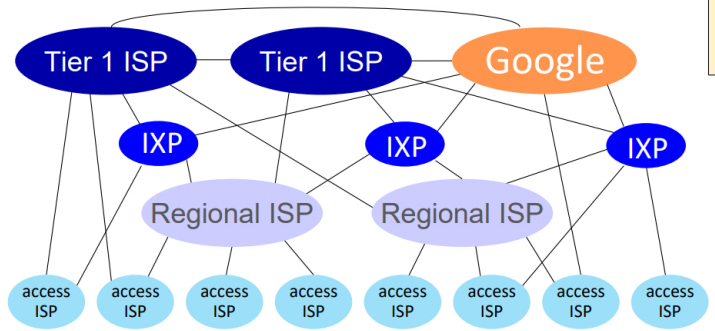
과도하게 데이터를 사용해야 하면 buffer overflow가 나서 딜레이와 손실이 생길 수 있음.

Internet structure : a "Network of networks"

host는 access ISPs를 통해서 인터넷에 접속한다.
ISP끼리 연결이 안 돼 있으면 멀리까지 접속할 수 없다.
네트워크의 네트워크는 복잡하다.

Q) 수백만개의 access ISPs 어떻게 연결하지?

한 개씩 다 연결하면 $O(N^2)$ 가 된다. 너무 복잡하다



Access ISP : 소규모 ISP. host가 인터넷에 연결하는 입구.

Global ISP : 대규모 ISP. 국제적인 네트워크 연결을 관리. 전 세계적으로 데이터를 송수신.

IXP : Internet Exchange Point. 여러개의 (Global) ISP를 연결해줌

Peering Link : 2개의 ISP끼리 연결을 하는 것

Regional ISP : 지역 ISP. 조금 더 저렴.

Content Provider Network : 구글, MS, Akami등 대용량으로 데이터를 처리하는 경우 자체적으로 사설 운영
사용자와 더 가깝게. 다른 ISP를 건너뛰고 바로 사용자와 연결될 수 있음.

Tier-1 commercial ISP : 국가 & 국가간 coverage ex)AT&T, Sprint, NTT

3강 실습- 웹페이지 퍼블리싱

Advertised speeds vs. Actual speeds

Advertised speed : 500Mbps, 1Gbps..

Actual speed: 광고보다 느림...

-> 왜 최고속도를 낼 수 없을까?

Bit (b): Basic unit of digital information (0 or 1)

Byte (B): 1 Byte = 8 bits

• $500 \text{ Mbps (megabits per second)} \div 8 = 62.5 \text{ MB/s (megabytes per second)}$

인터넷 속도에 영향을 미치는 요인들

- Network overhead or congestion - Peak times, Shared bandwidth
- Hardware limitations
- Routers and Modems, Device Performance, Cable quality - CAT.6 Max 1Gbps CAT.8 25Gbps
- Server and website limitations

Localhost (Loopback address)

- Localhost refers to **your own computer** within a network
- It uses the loopback network interface for **self-communication**
- **127.0.0.1** (IPv4); **::1** (IPv6); **localhost**
- Why do we need? Development; Testing; Security

4강 수업 - Application Layer

Network Security

네트워크 보안

인터넷은 원래 그다지 보안에 염두에 두고 만들어지지 않았음.

모든 layer에 보안에 대한 조치를 해야 한다.

어떻게 컴퓨터 네트워크가 공격당하는지, 어떻게 공격을 방어하는지, 공격에 면적이 되기 위해 어떻게 디자인해야 하는지를 염두해야 한다!

Packet Sniffing

Broadcast되는 정발(공유된 이더넷, 무선)

통과하는 모든 패킷을 읽고 기록함.

IP Spoofing

잘못된 source adress를 포함한 패킷을 injection함.

Denial of Service(DOS)

엄청난 리소스를 압도해서 리소스를 사용할 수 없게 하는 것.

1. 타겟을 정한다.
2. host 주변에 botnet을 만든다.
3. 봇들로부터 타겟으로 동시에 패킷을 보내게 함.

Layering

계층화

시스템의 구성 요소를 식별하고 그 관계를 명확히 할 수 있음

모듈화로 유지보수, 업데이트(maintenance, update)가 쉬워짐.

계층 내부에서 변화가 있어도 나머지 시스템은 영향을 받지 않음

>>시스템의 복잡성을 줄이고, 유지보수, 업데이트를 쉽게 만들어 줌.

Layered Internet Protocol stack

계층화된 인터넷 프로토콜 스택

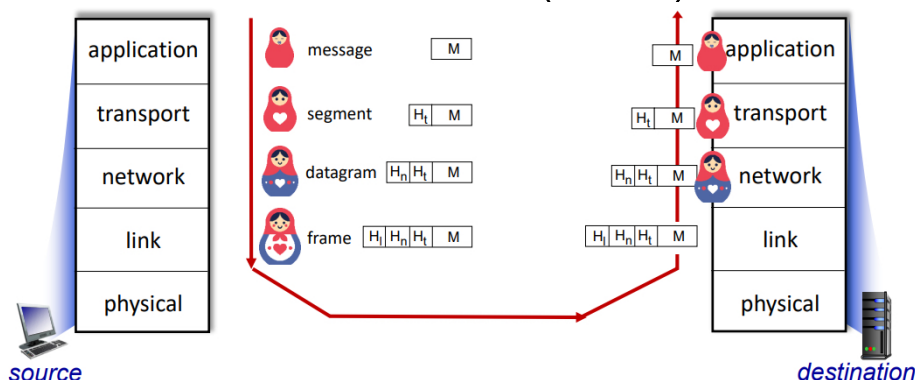
- | | | |
|----------------------|------------------------------------|------------------------|
| * application | 네트워크 어플리케이션을 지원 | HTTP, IMAP, SMTP, DNS |
| * transport | 프로세스 간 데이터 신뢰성있게 전송. 쪼갬다. 오류 검출. | TCP/UDP |
| * network | Source에서 Dest로 라우팅. 네트워크 내 패킷이동 관리 | IP, Routing Protocol |
| * link | 인접한 네트워크 장치 간 데이터 전송 | 이더넷, 802.11(WiFi), PPP |
| * physical | bit 단위 데이터 전송 | 케이블, 네트워크 카드, 허브... |

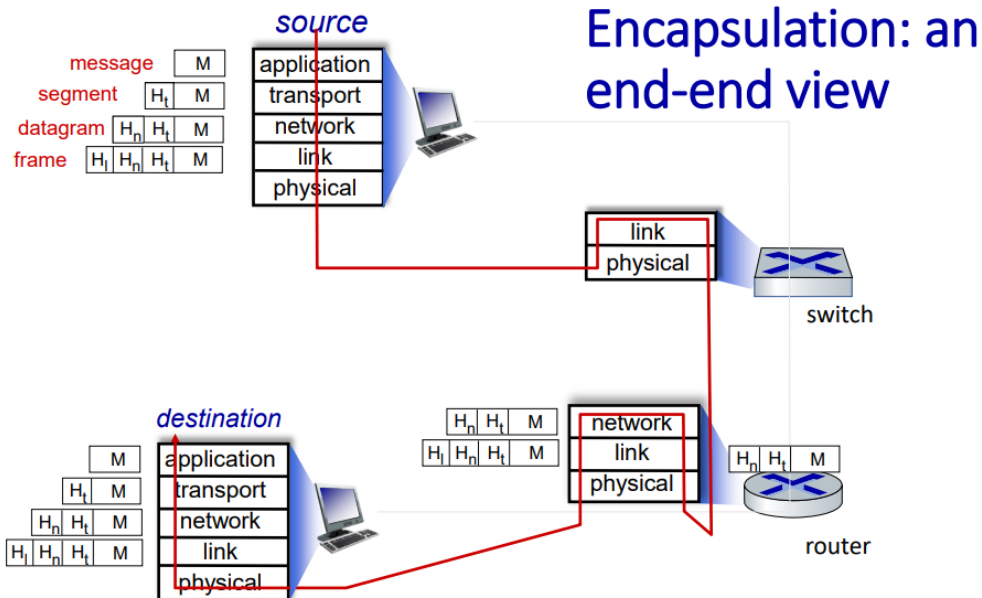
encapsulation : 한 단계 내려갈 때마다 데이터에 헤더를 덧붙여 감싸는 과정

H_t : transport 단계에서 붙는 헤더에는 몇 번째 메시지인지에 관한 내용

H_n : network 단계에서 붙는 헤더에는 어디서 어디로(어떤 호스트로) 가는지에 관한 내용

H_l : link 단계에서 붙는 헤더에는 정확히 누구한테(MAC 주소) 보내야 하는지에 관한 내용





6강 수업 - Application Layer 2

Application Layer

run on **end systems**

네트워크-코어 디바이스에 대해 생각할 필요 없음.

Application 단계에서는 메시지만 신경쓰면 됨!

빠른 앱 개발, 전파 가능.

일대일로 메시지만 주고받는 것처럼 느껴짐.

HTTP, SMTP, IMAP, DNS, 동영상 스트리밍 서비스.

Process Communicating

process : host 안에서 실행하는 프로그램

같은 host에서 두 프로세스들의 communication은 inter-process communication이라고 한다.(OS에 의해)

다른 host간의 프로세스 communication은 **메시지 교환**으로 일어난다.

client process : 통신을 시작하는 프로세스 ex) 카카오톡

server process : 접속을 기다리는 프로세스 ex) 카카오 서버

Sockets

애플리케이션과 네트워크 사이에서 **중개 역할**

앱 개발자는 socket 위 과정까지 개발하고 그 아래는 신경 쓸 필요가 없음

프로세스는 소켓으로부터 메시지를 받거나 전송한다.

메시지를 소켓으로 전달하기만 하면 됨. 하위 과정은 신경 쓸 필요 없음. - 우편 서비스

sender receiver 각 사이트에 socket이 하나씩 있어야 함. 총 2개!

Addressing Process

identifier(식별자)가 필요.

host 장치는 unique한 32비트 IP 주소를 가지고 있음.

>> but, IP만으로는 식별할 수 없음.

>> 따라서 **IP와 Port number**가 같이 있어야 함.

포트 넘버 예시 : HTTP서버 - 80 , 메일 서버 - 25

ex) HTTP 메시지를 보내려면 : IP 주소(128.119.245.12) + 포트 넘버 (80)을 같이 보내야!

Application-layer 프로토콜 정의

메시지 교환의 종류 : ex) request, response

메시지 syntax(문법) : 어떤 필드에 어떤 내용이 있고, 필드가 어떻게 구분되는지

메시지 semantics(의미론) : 각각의 필드가 무슨 의미인지

규칙(rules) : 언제 어떻게 프로세스가 보내지는지, 메시지에 답장하는지

공개 프로토콜 : RFC에 의해서 정의됨. 누구나 이용가능. ex) HTTP, SMTP

독점(소유) 프로토콜 : ex) Skype, Zoom...

Web and HTTP

web : 웹페이지.

객체(objects)들로 이루어짐. 객체는 HTML 파일, JPEG 사진, Java Applet, 음성...

웹페이지는 **base HTML 파일**로 구성되어 있음. 몇몇의 참조된 객체, 그리고 각각의 URL

URL(Uniform Resource Locator) : www.someschool.ecu/someDept/pic.gif

-----host name----- ---path name-----

HTTP

Hyper Text Transfer Protocol

Web의 Application-layer protocol

클라이언트 : request, receive해서 웹 객체를 띄움

서버 : 웹 서버는 요청받아 응답할 객체를 보냄.

HTTP Request message 요청

HTTP 메시지에는 두 종류가 있다 : request, response

HTTP request message는 ASCII 코드로 이루어짐.

Carriage Return \r 커서를 맨 끝으로

Line Feed \n 새 줄

request 메시지에서 처음 \r\n이 나오면 **request line** 끝

그 다음에 \r\n \r\n 이렇게 두 번 나오면 **header line** 끝

request 메시지에는 body가 거의 안 담겨 있음.

- ASCII (human-readable format)

```
request line (GET, POST, HEAD commands) → GET /index.html HTTP/1.1\r\n
header lines → Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel
Mac OS X 10.15; rv:80.0) Gecko/20100101
Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
carriage return, line feed at start of line indicates end of header lines → \r\n
carriage return character
line-feed character
```

* Check out the online interactive exercises for more

Request 메서드 종류

POST method(user input 1) : form 입력 전달 시. body에 user input이 담겨져서 전송됨.

GET method(user input 2) : URL 필드에 user data를 담아 request. ? 뒤에.

HEAD method : GET Method에서 object를 받아오지 않고 헤더 파일만 필요할 때. not body

PUT method : 새로운 파일을 업로드 할 때

HTTP Response message

응답

status line (protocol
status code status phrase)

header
lines

body, e.g., requested
HTML file

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
       OpenSSL/1.0.2k-fips PHP/7.4.9
       mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8

\r\n
<html> hello, cau ,
data,....

</html>
```

status line에서는 요청에 대한 처리 결과를 나타냄. 서버->클라이언트 응답 메시지.

200 OK

성공.

301 Moved Permanently

요청한 객체가 옮겨짐.

400 Bad Request

요청 메시지를 서버가 이해할 수 없음

404 Not Found

서버에서 요청한 문서를 찾을 수 없음

505 HTTP Version Not Supported

body에는 요청한 내용들이 담겨져 나옴. such as HTML 파일

4,6강 실습 - Developer Tools

Git

Version Control System (VCS)

타임캡슐과 비슷.

Tracks changes in source code during software development

변화를 추적.

Version Tracking; Branching and Merging; Collaboration

협업시 사용.

GitHub

- web-based platform for **hosting Git repositories**
- Remote Repositories; Collaboration Tools; Community
- Not the Same as Git: **Git is the tool; GitHub is the service**

Git 실습

git init : 폴더에 git을 초기화

git add . : 변경사항 추가

git commit : 변경사항 커밋

git push :

웹브라우저

WWW에서 정보를 접근, 회수, 볼 수 있는 소프트웨어 앱

유저와 웹 서버의 중개인

웹 기술들을 해석하고 렌더링함. - HTML, CSS, JS...

크롬, 사파리, 엣지 등....

유저가 URL 입력 -> DNS 검색 -> HTTP/HTTPS 요청을 서버로 보냄 -> 응답을 받음 -> 해석(렌더링) -> 표시

크로미움 프로젝트

구글이 시작하고 유지하고 있는 오픈소스 웹 브라우저 프로젝트

빠르고, 안정적이고, 안전한 웹을 만드는 것이 목표.

크로미움의 특징

오픈소스

웹 표준 준수

Cutting-edge 기술(세련됨)

가볍고 최적화된 퍼포먼스

크롬, 엣지, 오페라, 웨일 등이 크로미움 기반으로 제작됨.

• Chromium:

- Open-source project
- No proprietary Google features or branding
- No automatic updates
- Limited media codec support

• Chrome:

- Commercial browser by Google
- Automatic updates
- Integration with Google services
- Additional media codec support

크로미움과 크롬의 차이점 ----->

크롬 개발자 도구

웹페이지와 앱을 들여다보고, 디버깅, 분석하는데 사용됨

Element Panel : Element를 들여다볼 수 있음(Inspect)

HTML 수정 가능

Network Panel : **Monitor Requests**: See all network requests made by the page

Analyze Load Times: Identify slow resources

Inspect Headers: View request and response headers for debugging

Main Panels Overview

- **Elements**: Inspect and edit HTML and CSS
- **Console**: View error messages and run JavaScript commands
- **Sources**: Find the source file and debug JavaScript code
- **Network**: Monitor network requests and load times
- **Performance**: Analyze runtime performance
- **Application**: Manage storage, cookies, and cache
- **Security**: Check security protocols and certificates

7강 수업

Transport service가 App End에서는 무엇을 하는가?

Data Integrity 데이터 무결성

몇몇 앱들은 100프로 신뢰할 수 있는 데이터 전송을 요청한다

다른 몇몇 앱들은 일부 손실을 허용(toleration)한다.

ex) 파일 전송, 웹 통신

ex) 오디오...

Timing 타이밍

몇몇 앱들은 효율성을 위해 낮은 딜레이가 필요하다.

ex) 인터넷 전화, (슈팅) 게임, Xbox cloud

Throughput 처리량 : 시간당 처리되는 양

몇몇 앱들은 효율성을 위해 최소한(minimum)의 처리량이 필요하다. ex) 멀티미디어, 유튜브

throughput이 상관없는 앱

ex) elastic app

Security

encryption, data integrity

Internet transport protocols services 인터넷 전송 프로토콜 서비스

TCP service

reliable transport 송수신 프로세스 간 신뢰할 수 있음

flow control 송신자가 수신자를 overwhelm하지 않음

congestion control : 네트워크가 과부하(overload)되면 스로틀(throttle)을 한다.

connection-oriented : 연결 지향적. 클라이언트-서버 프로세스의 설정이 필요함 <-> 우편

does not provide : 타이밍, 최소 처리량(minimum throughput) 보장, 보안 지원 안함.

UDP service

unreliable data transfer : 송수신자 간 신뢰할 수 없음. reliable시 overhead가 있기 때문

does not provide : reliability, flow control, congestion control, timing, throughput guarantee, security, connection setup

>> UDP는 왜 존재하는 것일까? : loss를 허용하면서 빠르고 많은 전송이 필요할 때. 실시간 게임, 영상, 채팅...

HTTP overview

HTTP : HyperText Transfer Protocol

웹의 application-layer protocol

client/server 모델 :

client : requests, receive를 하는 브라우저, 웹 objects를 “표시(displays)”

server : web server는 requests에 대한 응답으로 objects들을 보낸다.

HTTP use TCP

HTTP는 TCP를 사용한다!

1. client는 서버에 TCP 연결을 **시작**(initiates)한다.(소켓을 생성) 포트 80번
2. 서버가 클라이언트로부터 TCP 연결을 수용한다(accepts)
3. HTTP 메시지(app layer 메시지)를 브라우저(client)-웹서버(server) 간 교환한다.
4. TCP 연결 종료(closed)

HTTP is "stateless"

서버는 과거에 클라이언트가 어떤 요청을 했는지 정보를 유지하지 않음(기억 못함)

<-----> “state”를 유지하는 프로토콜은 복잡하다(만들기 어렵다)

과거 기록이 유지되어야 하는 프로토콜

서버/클라이언트가 충돌 등을 하면, "state(상태)"가 불연속적이게 되고, 해결해야만 한다.

HTTP connections: two types

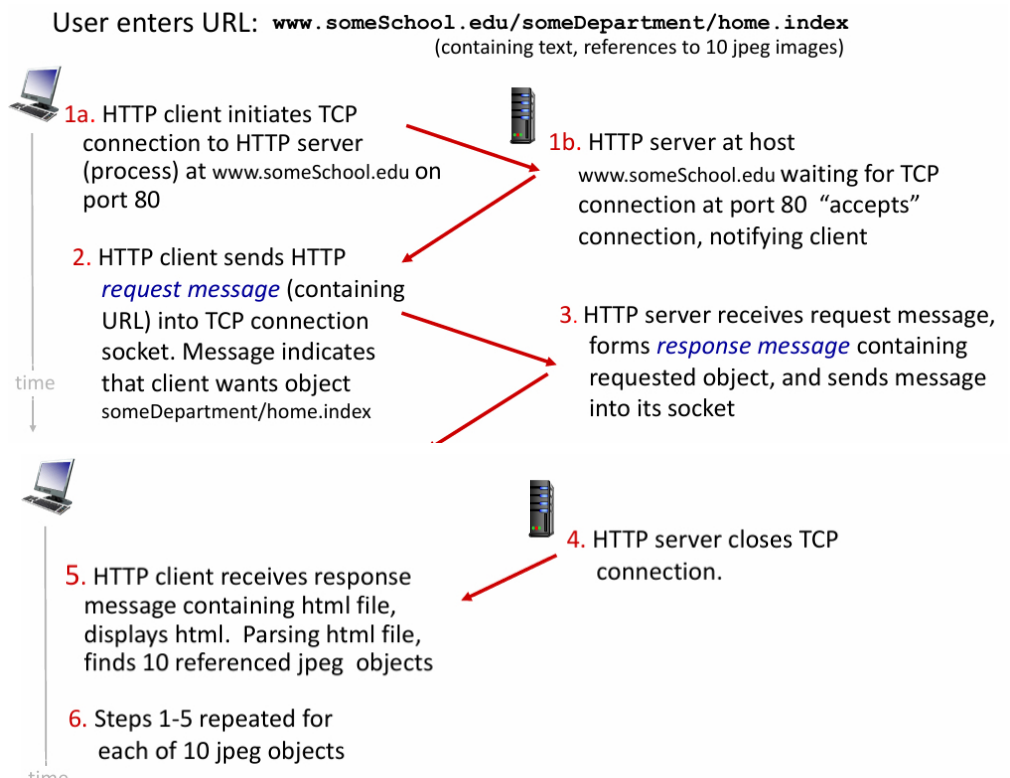
Non-persistent HTTP 비지속적인 HTTP

1. TCP 연결 열림
 2. 한번에 하나의 object만 주고 받음
 3. TCP 연결 닫힘
- >> 여러개의 object를 보내려면 여러번 연결을 해야 함.

Persistent HTTP 지속적인 HTTP

1. TCP 연결 열림
2. 한번 열리면, 하나의 TCP 연결로 여러개의 object를 클라이언트/서버 간 전송할 수 있음
3. TCP 연결 닫힘

Non-Persistent HTTP 예시



RTT Round Trip Time 왕복시간

: 작은 패킷이 클라이언트->서버->클라이언트를 하는 1번 왕복 시간

HTTP respond time(per object)

* TCP 시작하는데 한 RTT가 필요함.

* HTTP request하고 처음 도착하는 몇 개의 bytes 응답까지 시간이 필요함. 이때까지 또 한 RTT 필요함.

>> Non-persistent HTTP respond time : 2RTT + 파일 전송 시간

Persistent HTTP

Non-per HTTP의 문제점

오브젝트 당 2RTT가 필요함.

OS overhead가 각각의 TCP 연결마다 발생함.

여러개의 TCP 연결을 병렬적으로 하는 경우도 있음 but 일단은 serialize 하게 연결한다고 생각하기!

Persistent HTTP

서버가 응답을 한 이후에 연결을 열어둠

이어지는 메시지도 같은 연결로 보낼 수 있음

오브젝트가 요청(reference)되자마자 보낼 수 있음

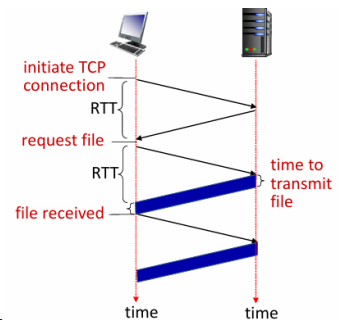
각각의 요청된 오브젝트(referenced object)당 1RTT만 있으면 된다. => 시간이 반으로 줄어듦.

>> Persistent HTTP respond time : (처음에만 RTT) + RTT + 파일 전송 시간

>> HTTP-Respond time

Non-per = 10*(2RTT + File transmission time)

Per = RTT + 10*(RTT+File Transmission time)



Cookie 쿠키 : User/Server의 상태 저장

: 웹사이트와 클라이언트 브라우저는 통신에서의 상태를 저장하기 위해서 쿠키를 쓴다.

4가지 component

1. HTTP response 메시지를 저장하는 cookie header line
2. cookie header line에는 다음 HTTP request message가 있음
3. cookie file은 user's host에 저장되고 user's browser에 의해 관리됨
4. 웹사이트의 백엔드 DB에 쿠키 내용이 저장되어 있음

예시)

수잔이 노트북으로 e-커머스 사이트에 처음 방문했다.

첫 HTTP request가 사이트에 도착하면 사이트가 아래 내용을 생성한다.

* unique한 ID(aka Cookie)

* 그 ID에 대한 정보를 백엔드 DB에 저장함

연속적인(다음) HTTP request에는 수잔이 Cookie ID값을 가지고 있으므로 사이트가 수잔임을 식별할 수 있음

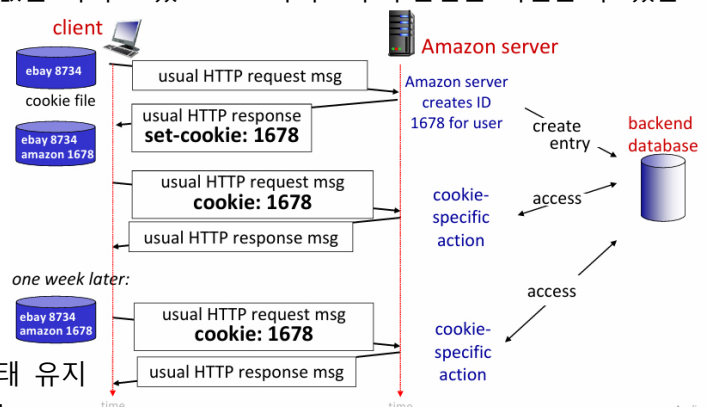
용도)

인증(자동 로그인)

장바구니

추천

유저 세션 유지(웹 메일)



어떻게 상태(state)를 유지하는가?

protocol endpoint : 송수신자의 여러 통신동안 상태 유지

메시지에서 : HTTP 메시지에서 쿠키를 사용해 유지

>> 쿠키와 프라이버시 : 쿠키는 내가 누군지 알려줌. 서드파티 지속 쿠키는 여러 사이트를 걸쳐다니며 추적함.

GDPR과 쿠키

쿠키가 개인을 식별할 수 있다면, 쿠키를 개인정보로 간주해야하고 GDPR 규제를 따라야 한다. GDPR에 따라서 사용자는 명시적으로 쿠키 사용을 허용/거부할 수 있다.

E-mail

HTTP는 쿠키를 사용하지 않으면 기본적으로 "Stateless(기억 못함)"이다.

<-> 이메일은 "State"

이메일의 세 가지 요소

- * user agents 유저 에이전트
- * mail servers 메일 서버
- * SMTP(Simple Mail Transfer Protocol) : 메일 서버(회사) 간 연결

1. User Agent(사용자)

aka Mail Reader

메일 메시지를 구성하고, 편집하고, 읽는다.

ex) Outlook, iPhone mail client

들어가고 나오는 메시지가 서버에 저장됨

2. Mail servers(서버)

mailbox : user의 수신된 메시지를 저장. 개인 메일함

message queue : 전달되어야 하는(outgoing) 메시지의 queue

3. SMTP protocol

메일 서비스들 간 메시지를 보내는 프로토콜

clinet : 보내는 메일 서버

server : 받는 메일 서버

SMTP RFC

TCP를 사용함. 신뢰할 수 있는(reliable) 메시지 전송을 위해서 포트 25번!

direct transfer : sending server와 receiving server가 직접적으로 전송한다.

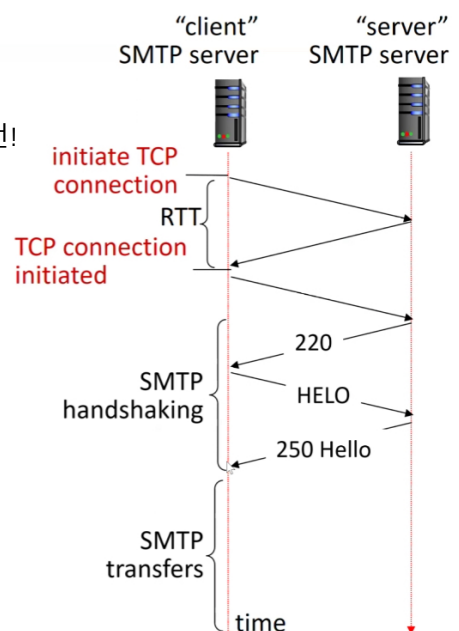
전송 3단계

- 0.(전송 전) TCP 연결.
- 1. SMTP Handshaking : 서로를 인식. 전송 준비가 되었음을 확인
- 2. SMTP Transfer of messages : 메시지 전송
- 3. SMTP closure : 닫기

Command/Response Interaction(Http와 비슷)

command : ASCII text

```
response : Status code and phrase    ???
```



예시1) 메일 보내는 프로세스 PPT 참조 | 예시2) SMTP 메시지 코드 있는 페이지 PPT 참조

HTTP vs SMTP

HTTP : client pull 클라이언트가 정보를 땡겨옴(받아옴)

SMTP : clinet push 클라이언트가 정보를 밀음(보냄)

한 메시지에 한 오브젝트가 캡슐화되어 있음
multipart 메시지에 여러 object를 담을 수 있음
(사진, 텍스트 등등)

SMTP는 Persistent Connections 사용. 메시지가 7비트 ASCII로 되어야 함 메시지가 끝날 때 "."으로 끝냄

>> 둘 다 ASCII 명령/응답 사용. Status Code 사용

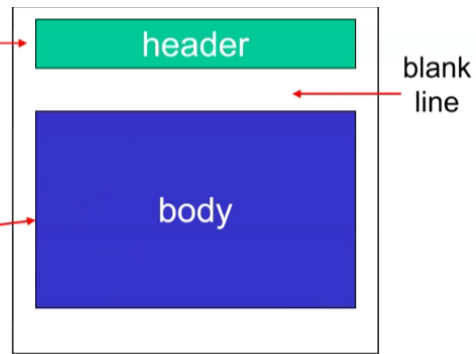
메일 메시지 포맷 Mail message Format

SMTP : RFC 5321에 규정되어있는 메일교환 프로토콜

RFC 2822에 이메일 메시지를 위한 syntax가 규정되어 있음(HTML과 유사)

header(전송자, 수신자, 제목)와 body로 구분

- header lines, e.g.,
 - To:
 - From:
 - Subject:these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the "message" , ASCII characters only



DNS(Domain Name Service)

Distributed DB에 수많은 Name Servers(이름 서버)의 hierarchy가 구현되어 있다.

Application-layer protocol : host-DNS-server 간의 통신을 이름으로 해결해준다.

Core-Internet function임.

Application-layer Protocol로 구현되어 있음.

네트워크 엣지의 복잡성???

DNS service

hostname-IP 간 번역!

host aliasing : canonical(진짜 이름), alias names(우리가 알고 있는 약어)

black.nm.cau.ac.kr ns.cau.ac.kr

mail server aliasing : 메일 서버 이름도 aliasing을 함

load distribution 로드 분산 : 한 이름이지만 실제 접속은 여러 서버로 분산

7강 실습

HTTP Status Codues

3자리로 이루어진 클라이언트의 요청에 대한 서버의 응답

200 OK 통신이 정상적으로 성공

304 Not Modified 새로 로드하는 경우, 수정되지 않은 파일. 캐시에 남아있다가 다음에 로드할 때 빠른 로딩 가능

404 Not Found (클라이언트) 오류

500 Internal Server Error

GET, POST만 사용 예정

GET : 서버로부터 받아옴 - 맨 처음에는 물음표(?) 그 이후에는 & 로 잇기

POST : 서버로 보냄 -

PUT : 기존 내용(리소스) 업데이트

DELETE : 서버에 있는 리소스 삭제

GET

URL/?name=심성민&major=산보

4. 응답 예시

POST

XML	http://kobis.or.kr/kobisopenapi/webservice/rest/movie/searchMovieList.xml?key=82ca741a2844c5c180a208137bb92bd7
JSON	http://kobis.or.kr/kobisopenapi/webservice/rest/movie/searchMovieList.json?key=82ca741a2844c5c180a208137bb92bd7

7강 학생발표

소켓 : 네트워크 통신의 끝점(endpoint)

소켓 주소 : IP주소 + 포트주소

특징	Stream Socket	Datagram Socket
프로토콜	TCP(연결 지향형)	UDP(비연결 지향형)
연결 설정	연결을 설정 후 데이터 전송	연결 없이 데이터 전송
데이터 전송 단위	연속적인 바이트 스트림	개별 데이터그램
신뢰성	데이터의 신뢰성과 순서 보장	신뢰성과 순서 보장 없음
전송 속도	비교적 느림(오버헤드 큼)	빠름(오버헤드 작음)
사용 예시	HTTP, FTP, 이메일	스트리밍, 온라인 게임

TCP 3-Way Handshaking

1. 클라이언트 -> 서버 SYN 보냄
2. 서버 -> 클라이언트 SYN/ACK 보냄
3. 클라이언트 -> 서버 ACK 보냄

TCP 4-Way Handshaking 안전하게 종료하기 위한 방법

1. 클라이언트 -> 서버 종료요청 FIN보냄
2. 서버 -> 클라이언트 ACK 보냄
3. 서버 -> 클라이언트 해제할 준비가 끝나면 FIN 보냄
4. 클라이언트 -> 서버 해제 준비 끝나면 FIN 보냄.

웹브라우저의 역사

팀 버너스리 - WWW 개발. 페이지 구분은 URL로, 페이지 언어는 HTML로

마크 안데르센 - Mosaic 최초의 이미지+텍스트 보여주는 브라우저

1차 브라우저 전쟁

넷스케이프 : 마크 안데르센 x 제임스 클락가 만듦.

익스플로러 : 넷스케이프와 경쟁. 신기능 탑재. 윈도우에 선탭재

2차 브라우저 전쟁

크롬 : 가벼운 브라우저. 현재 1위

웹페이지 vs 웹사이트

웹페이지 : 개별 사이트

웹사이트 : 웹페이지의 집합체

HTML : HyperText Markup Language - 웹페이지의 구조

<html> <head> <body>

JS : 웹페이지의 기능.

동적컨텐츠 생성, 이벤트 처리, 애니메이션

CSS : 웹페이지의 스타일 레이아웃

디자인&레이아웃, 반응형 디자인(기기별), 애니메이션, 글꼴

Web Font : 폰트 -호스트에 기반하여 이미 다운로드 되어있음. <-> 웹폰트 - 접속시 서버에서 다운로드

플랫폼의 역사

80s 개인용 컴퓨터가 인기를 얻음. 윈도우/맥

90s 구글, 야후같은 검색 엔진이 주류가 됨

00s 안드로이드 iOS로 이동중에도 플랫폼에 접근가능하게 됨.

Environment 환경

다양한 활동을 할 수 있는 공간

Virtual space 가상공간

사용자가 연결, 공유할 수 있는 가상 공간

Connect 연결

다양한 서비스들과 연결, 상호작용

물리적 공간

하드웨어 플랫폼

서비스 플랫폼